

Calculation Flow Design

1. Objective

Calculation Flow คือขั้นตอนการคำนวณราคาสุทธิของคำสั่งซื้อ ตั้งแต่รับ Order เข้ามา จนได้ราคาสุดท้ายหลังหัก Promotion ทั้งหมด

เป้าหมายของ Flow นี้คือ:

- คำนวณราคาถูกต้อง
- Promotion หลายตัวต้องมีลำดับชัดเจน
- รองรับ Promotion ซ้อนกัน
- ป้องกันราคาติดลบ
- ตรวจสอบย้อนหลังได้ด้วย Calculation Breakdown
- เพิ่ม Promotion ใหม่ได้โดยไม่กระทบ Flow หลัก

2. Core Calculation Principle

ระบบต้องใช้แนวคิด **Deterministic Calculation**

หมายความว่า:

```
Input เดิม
Promotion Config เดิม
เวลาเดียวกัน
ต้องได้ผลลัพธ์เดิมเสมอ
```

ดังนั้นห้ามปล่อยให้ลำดับการคำนวณขึ้นกับลำดับที่ Database ส่งกลับมาแบบไม่แน่นอน

ต้องมีการ Sort ชัดเจน เช่น:

```
scope_order ASC
priority ASC
created_at ASC
promotion_id ASC
```

3. Recommended Calculation Order

ลำดับการคำนวณที่แนะนำคือ:

ITEM → CART → COUPON → SHIPPING → ROUNDING

Step	Scope	Description
1	ITEM	ลดราคาที่ระดับสินค้าแต่ละรายการ
2	CART	ลดราคาจากยอดรวมตะกร้าหลัง Item Discount
3	COUPON	ลดจาก Coupon Code
4	SHIPPING	ลดค่าส่ง เช่น Free Shipping
5	ROUNDING	ปัดเศษตาม Business Rule

เหตุผลที่ Item-level ต้องมาก่อน Cart-level:

Cart-level Promotion ควรอ้างอิงยอดรวมหลังจากลดระดับสินค้าแล้ว

ตัวอย่าง:

Product 1: 1,000 - 10% = 900

Product 2: 500 - 100 = 400

Cart Subtotal หลัง Item Discount = 1,300

Cart Discount จะคำนวณจาก 1,300

4. High-Level Calculation Flow

```
Receive Order Request
↓
Validate Request
↓
Load Product Price from Server
↓
Build Calculation Context
↓
Load Active Promotions
```

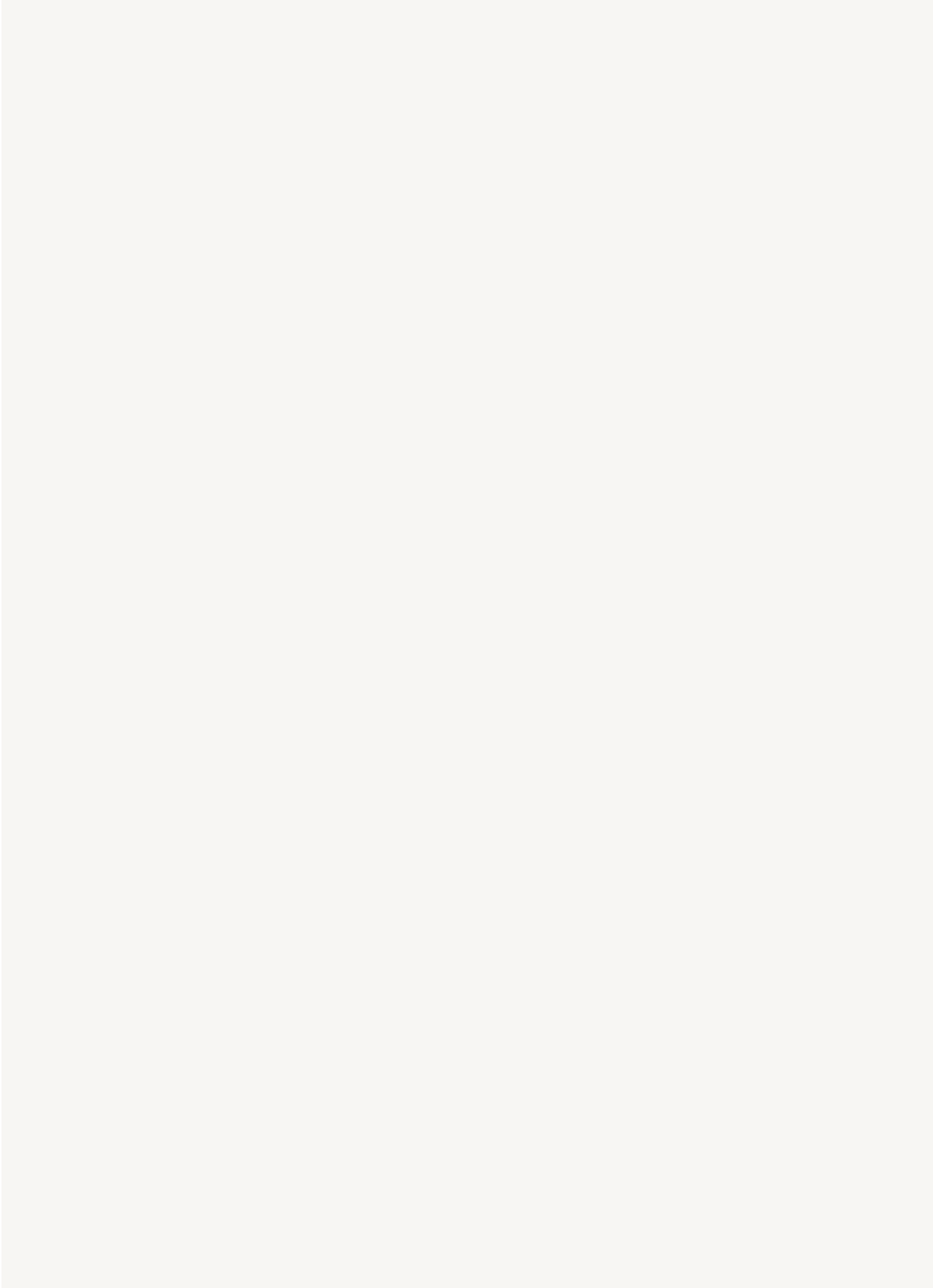
```
↓
Filter Eligible Promotions
↓
Sort Promotions
↓
Apply Item-level Promotions
↓
Apply Cart-level Promotions
↓
Apply Coupon Promotions
↓
Apply Shipping Promotions
↓
Apply Rounding
↓
Validate Final Amount
↓
Generate Calculation Breakdown
↓
Save Audit Log
↓
Return Final Price
```

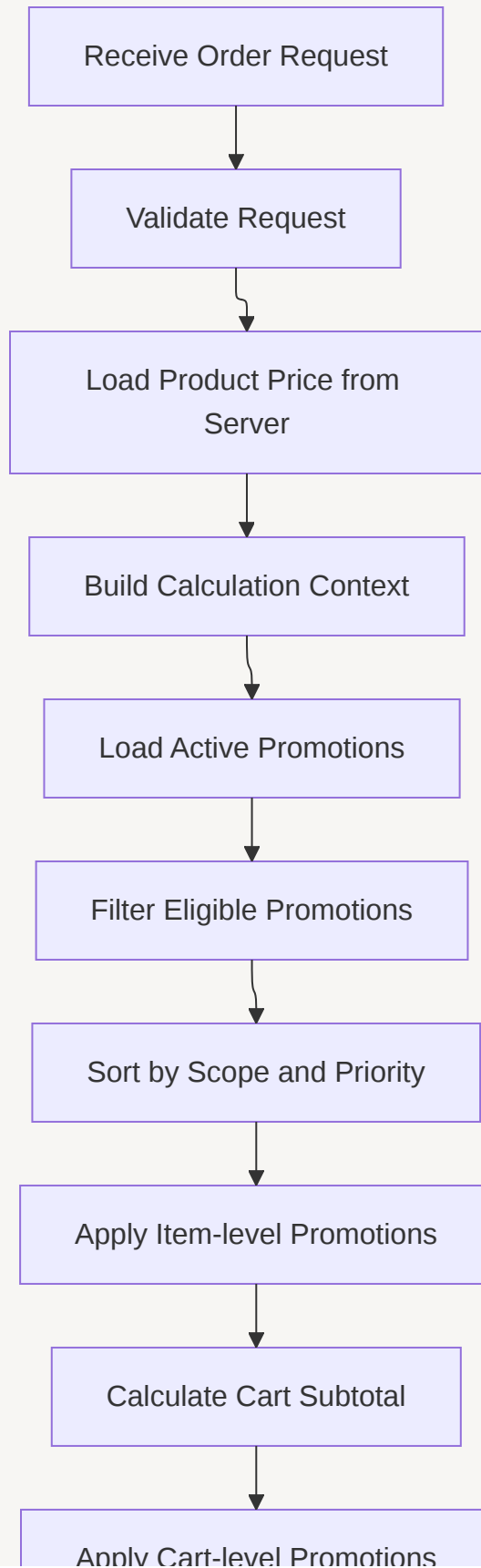
5. Flow Diagram

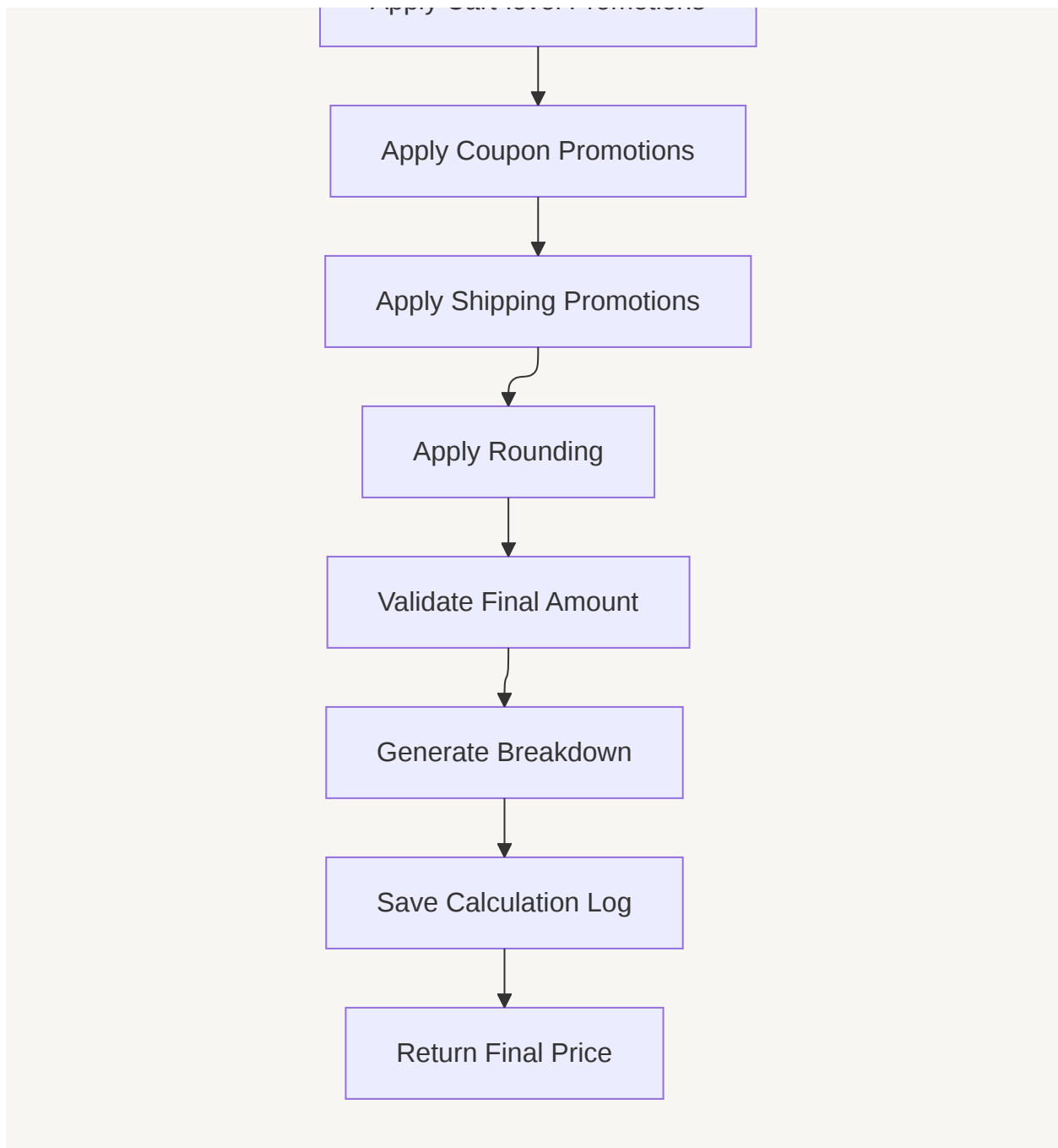
flowchart TD

```
A[Receive Order Request] --> B[Validate Request]
B --> C[Load Product Price from Server]
C --> D[Build Calculation Context]
D --> E[Load Active Promotions]
E --> F[Filter Eligible Promotions]
F --> G[Sort by Scope and Priority]
G --> H[Apply Item-level Promotions]
H --> I[Calculate Cart Subtotal]
I --> J[Apply Cart-level Promotions]
J --> K[Apply Coupon Promotions]
K --> L[Apply Shipping Promotions]
L --> M[Apply Rounding]
M --> N[Validate Final Amount]
N --> O[Generate Breakdown]
```

```
0 --> P[Save Calculation Log]
P --> Q[Return Final Price]
```







6. Detailed Step-by-Step Flow

Step 1: Receive Order Request

ระบบรับข้อมูลคำสั่งซื้อ เช่น:

```
{
  "user_id": 1,
  "coupon_code": null,
  "items": [
    {
      "product_id": 1,
      "quantity": 1
    },
    {
      "product_id": 2,
      "quantity": 1
    }
  ]
}
```

ข้อสำคัญ:

Client ไม่ควรส่ง price มาเป็น Source of Truth

ระบบต้องโหลดราคาจาก Database หรือ Product Service เอง

Step 2: Validate Request

ตรวจสอบข้อมูลพื้นฐาน:

- `items` ต้องไม่ว่าง
- `product_id` ต้องมีค่า
- `quantity` ต้องมากกว่า 0
- `coupon_code` ถ้ามี ต้องอยู่ในรูปแบบที่ถูกต้อง
- ห้ามมีสินค้าที่ระบบไม่รู้จัก
- สกุลเงินของสินค้าต้องตรงกัน

ถ้าไม่ผ่าน ให้หยุดก่อนเข้าสู่ Promotion Engine

Step 3: Load Product Price

ระบบโหลดข้อมูลสินค้าจาก Server-side Source เช่น Database

ข้อมูลที่ต้องใช้:

Field	Description
product_id	รหัสสินค้า
sku	SKU
category_id	หมวดหมู่สินค้า
unit_price	ราคาต่อชิ้น
currency	สกุลเงิน
status	ACTIVE / INACTIVE

ตัวอย่าง:

```
Product 1 = 1,000 บาท  
Product 2 = 500 บาท
```

ควรเก็บเงินจริงเป็นหน่วยเล็กสุด เช่น satang:

```
1,000 บาท = 100000 satang  
500 บาท = 50000 satang
```

Step 4: Build Calculation Context

สร้าง Object กลางสำหรับเก็บ State ระหว่างคำนวณ

ตัวอย่างข้อมูลใน Context:

```
user_id  
coupon_code  
items  
original_total  
discount_total  
final_total  
applied_promotions
```

```
skipped_promotions
stopped_scopes
```

แนวคิดสำคัญ:

ห้ามแก้ original price โดยตรง
ให้เก็บ discount และ final amount แยกต่างหาก

ตัวอย่าง Item Context:

```
product_id = 1
unit_price = 100000
quantity = 1
original_amount = 100000
discount_amount = 0
final_amount = 100000
```

Step 5: Load Active Promotions

โหลด Promotion ที่มีโอกาสใช้ได้เท่านั้น

เงื่อนไขเบื้องต้น:

```
status = ACTIVE
starts_at <= now
ends_at >= now
```

ควรรโหลด Promotion แบบ Batch พร้อมข้อมูลที่เกี่ยวข้อง:

- conditions
- actions
- targets
- stacking policy

ข้อควรระวัง:

ห้าม Query Database ใน Loop ของสินค้า

ไม่ดี:

```
for each item:  
    query promotions by product_id
```

ดี:

```
load active promotions once  
build in-memory map  
calculate in memory
```

Step 6: Filter Eligible Promotions

กรอง Promotion ที่ไม่เกี่ยวข้องออกก่อน

ตัวอย่าง Filter:

- Promotion หมดอายุ
- Promotion ยังไม่เริ่ม
- Promotion Inactive
- Coupon required แต่ request ไม่มี coupon
- Target ไม่ตรงสินค้า
- Condition ไม่ผ่าน
- Usage limit เต็ม

ผลลัพธ์ควรเก็บทั้ง:

```
eligible_promotions  
skipped_promotions พร้อม reason
```

ตัวอย่าง skipped reason:

```
expired
inactive
target_not_matched
condition_not_matched
coupon_required
usage_limit_reached
```

Step 7: Sort Promotions

Promotion ที่ผ่าน Eligibility แล้วต้องถูก Sort แบบ Deterministic

ลำดับที่แนะนำ:

```
scope_order ASC
priority ASC
created_at ASC
promotion_id ASC
```

Scope Order:

Scope	Order
ITEM	1
CART	2
COUPON	3
SHIPPING	4

Priority Rule:

```
priority น้อยกว่า = คำนวณก่อน
```

ตัวอย่าง:

```
Promotion A priority = 1
Promotion B priority = 10

Promotion A ต้องถูก Apply ก่อน Promotion B
```

Step 8: Apply Item-level Promotions

ใช้กับสินค้ารายการใดรายการหนึ่ง เช่น:

```
Product 1 ลด 10%  
Product 2 ลด 100 บาท
```

Flow:

```
for each item:  
    find matched item promotions  
    sort by priority  
    apply promotion one by one  
    update item discount  
    ensure final_amount >= 0
```

ตัวอย่าง:

```
Product 1:  
original = 1,000  
discount = 10% = 100  
final = 900  
  
Product 2:  
original = 500  
discount = 100  
final = 400
```

Step 9: Apply Cart-level Promotions

หลังจากคำนวณ Item-level แล้ว ให้รวมยอดใหม่เป็น Cart Subtotal

```
cart_subtotal = sum(item.final_amount)
```

ตัวอย่าง:

```
Product 1 final = 900
Product 2 final = 400

cart_subtotal = 1,300
```

ถ้ามี Cart Promotion เช่น ซื้อครบ 1,000 ลด 100:

```
1,300 - 100 = 1,200
```

Step 10: Apply Coupon Promotions

Coupon อาจถือเป็น Cart-level Promotion ที่ต้องมี Code ต้องตรวจเพิ่ม:

- coupon_code ถูกต้อง
- coupon ยังไม่หมดอายุ
- coupon ยังไม่ถูกใช้เกิน limit
- user ยังไม่ใช้เกิน limit
- coupon ใช้ร่วมกับ Promotion อื่นได้หรือไม่

ในขั้น Calculate:

```
ยังไม่ consume coupon usage
```

ในขั้น Confirm Order:

```
ต้อง re-calculate และ consume usage ภายใต transaction
```

Step 11: Apply Shipping Promotions

Shipping Promotion ใช้กับคำสั่ง เช่น:

```
shipping_fee = 50
free_shipping_discount = 50
shipping_final = 0
```

ไม่ควรนำ Shipping ไปปนกับ Item หรือ Cart Discount ถ้า Business ไม่ได้กำหนดไว้

Step 12: Apply Rounding

Rounding ควรทำท้ายสุด หรือทำตาม Business Rule ที่กำหนดไว้ชัดเจน

หลักสำคัญ:

ห้ามใช้ float สำหรับเงิน

ควรใช้:

BIGINT minor unit เช่น satang
หรือ Decimal library

ถ้าใช้ Percentage ที่มีทศนิยม แนะนำใช้ basis points:

```
10% = 1000 basis points
10.50% = 1050 basis points
100% = 10000 basis points
```

7. Stacking Flow

Promotion แบบซ้อนกันต้องควบคุมด้วย Field เหล่านี้:

Field	Meaning
priority	ลำดับการคำนวณ
stackable	ใช้ร่วมกับ Promotion อื่นได้หรือไม่
exclusive	ใช้แล้วห้ามตัวอื่นใน Scope เดียวกัน
stop_processing	ใช้แล้วหยุด Rule ถัดไป

Field	Meaning
scope	ขอบเขตของ Promotion

Case 1: Stackable Promotion

Product price = 1,000

Promo A:

ลด 10%

priority = 1

stackable = true

Promo B:

ลด 100 บาท

priority = 2

stackable = true

Calculation:

Start = 1,000

Apply Promo A: $1,000 - 10\% = 900$

Apply Promo B: $900 - 100 = 800$

Final = 800

Case 2: Exclusive Promotion

Product price = 1,000

Promo A:

ลด 10%

priority = 1

exclusive = true

Promo B:

ลด 100 บาท

priority = 2

Calculation:


```
Apply Promo A: 1,000 - 10% = 900
Promo A เป็น exclusive
Promo B ถูก skip

Final = 900
```

Case 3: Stop Processing

```
Promo A:
priority = 1
stop_processing = true

Promo B:
priority = 2
```

ถ้า Promo A Apply สำเร็จ:

```
Promo B จะถูก skip
reason = scope_stopped
```

Case 4: Non-stackable Promotion

ถ้า Promotion ไม่สามารถใช้ร่วมกับ Promotion อื่นได้ ต้องมี Policy ชัดเจน
ตัวเลือกที่ใช้ได้:

Option A: Priority First

Promotion ที่ Priority สูงกว่าได้ใช้ก่อน แล้วตัวอื่นถูก Skip
ข้อดี:

- ง่าย
- เร็ว
- Deterministic
- Debug ง่าย

ข้อเสีย:

- ลูกค้าอาจไม่ได้ส่วนลดมากที่สุด

Option B: Best Discount Wins

ระบบจำลอง Promotion ที่ขัดแย้งกัน แล้วเลือกตัวที่ให้ส่วนลดสูงสุด

ข้อดี:

- ลูกค้าได้ส่วนลดดีที่สุด

ข้อเสีย:

- ซับซ้อน
- คำนวณหนักกว่า
- Debug ยากกว่า

สำหรับ Version แรก แนะนำ:

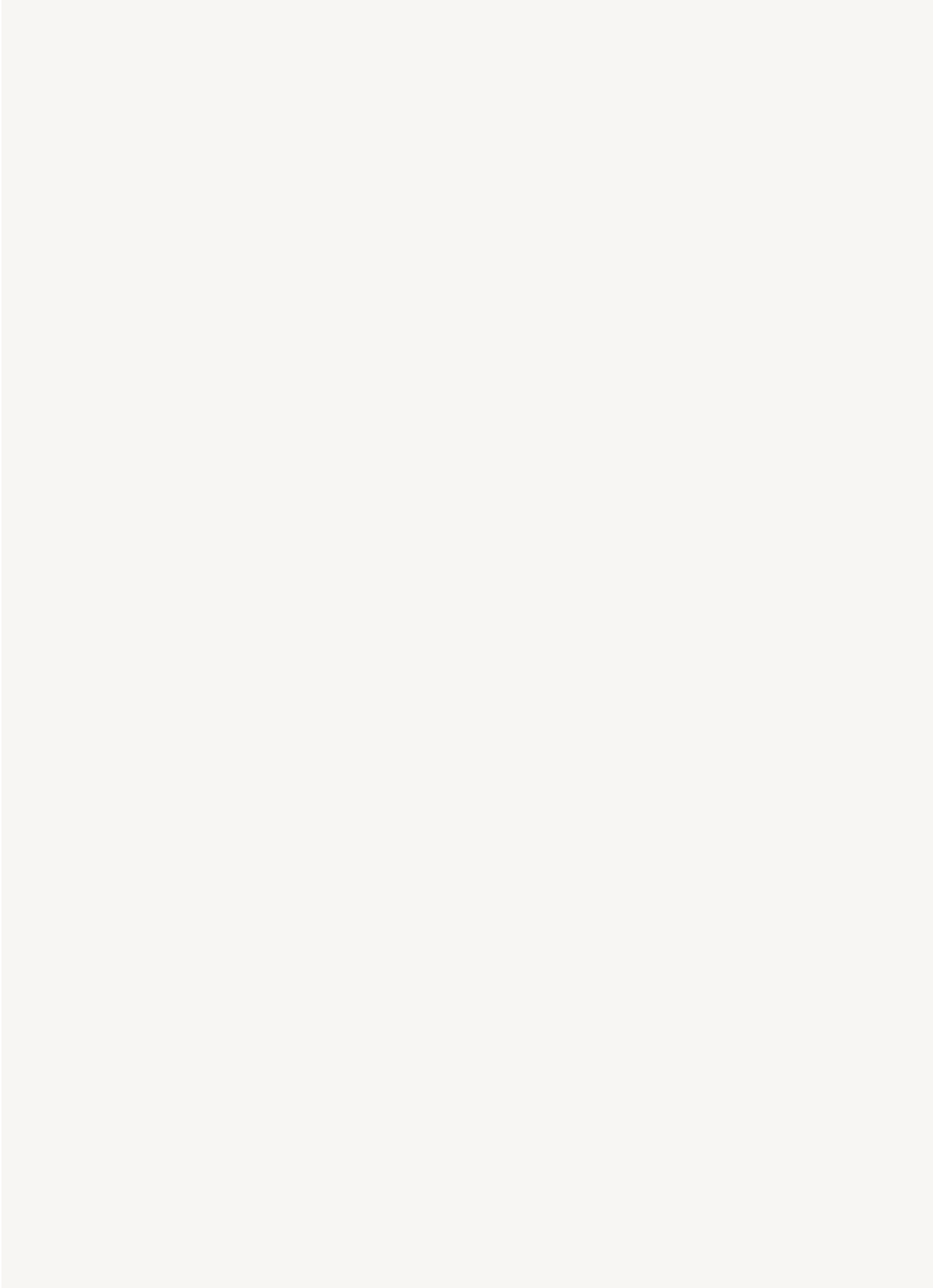
ใช้ Priority First Policy

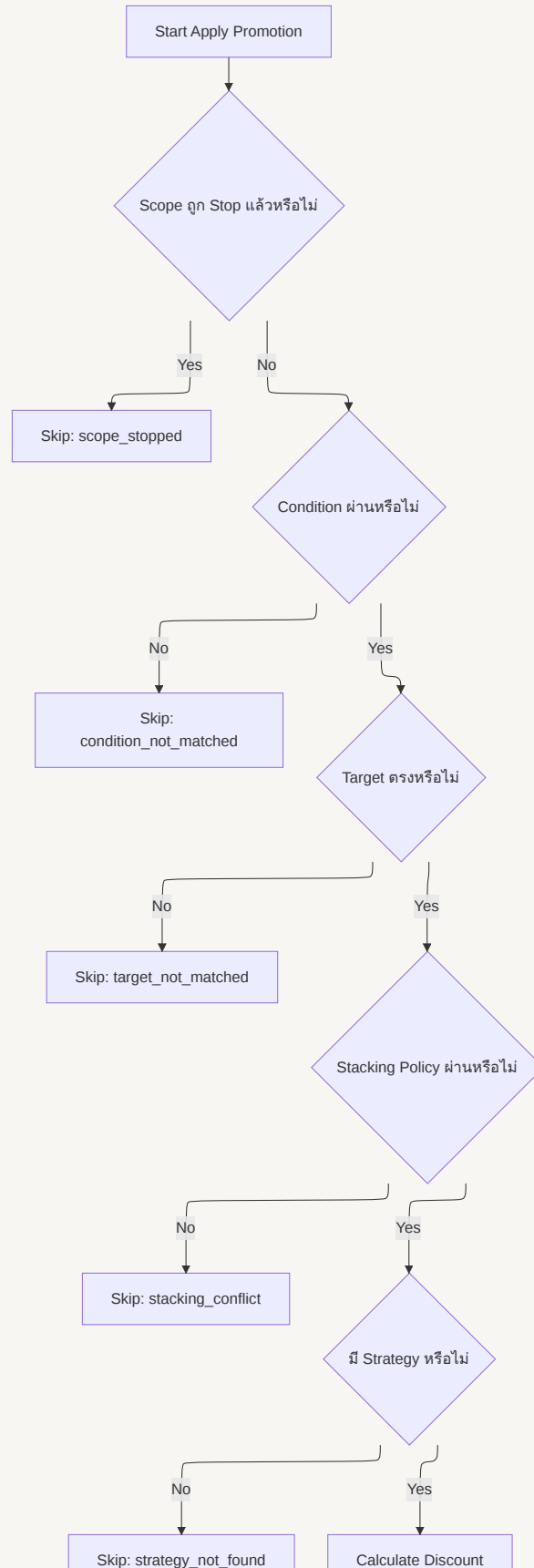
8. Detailed Decision Flow

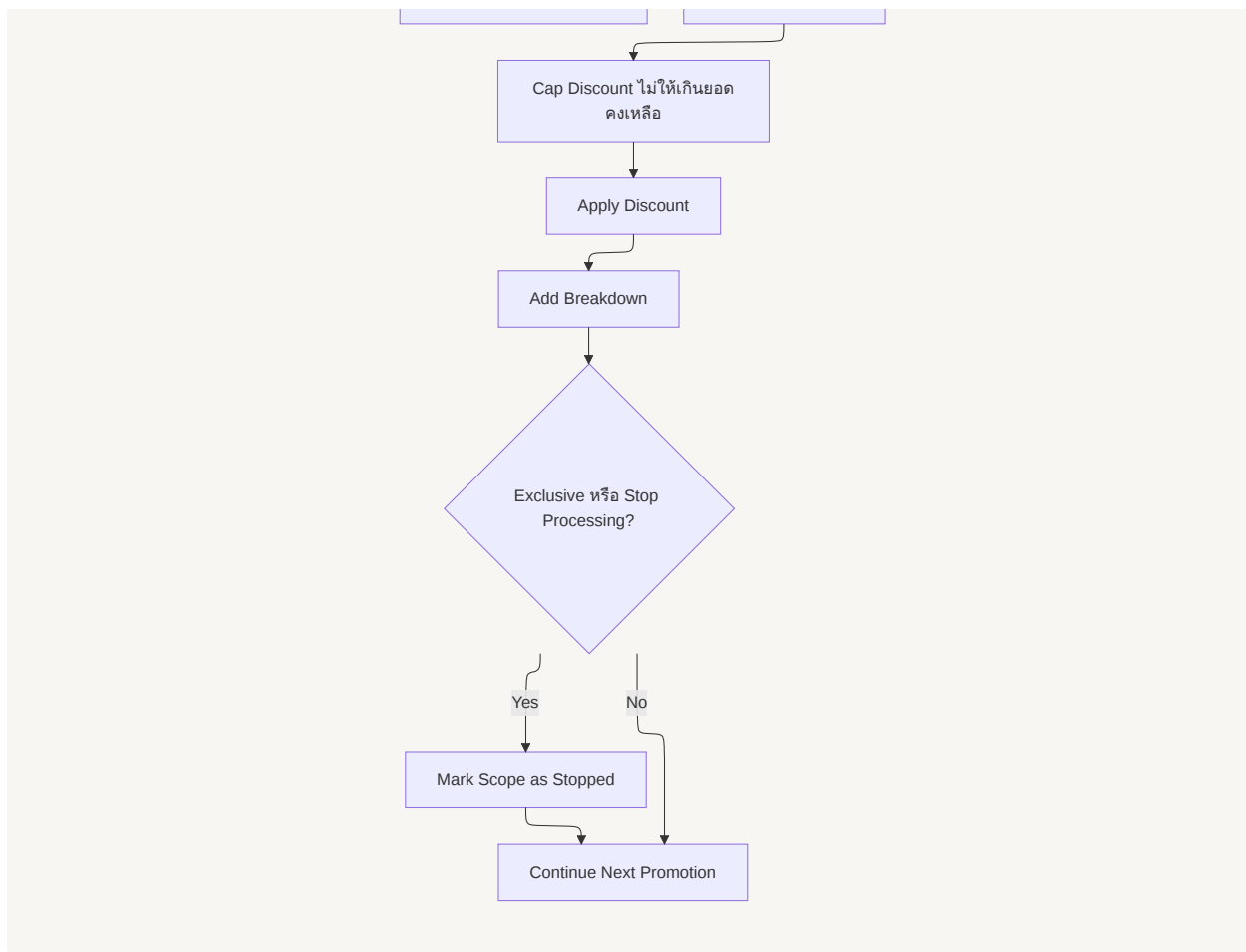
flowchart TD

```
A[Start Apply Promotion] --> B{Scope ถูก Stop แล้วหรือไม่}
B -- Yes --> C[Skip: scope_stopped]
B -- No --> D{Condition ผ่านหรือไม่}
D -- No --> E[Skip: condition_not_matched]
D -- Yes --> F{Target ตรงหรือไม่}
F -- No --> G[Skip: target_not_matched]
F -- Yes --> H{Stacking Policy ผ่านหรือไม่}
H -- No --> I[Skip: stacking_conflict]
H -- Yes --> J{มี Strategy หรือไม่}
J -- No --> K[Skip: strategy_not_found]
J -- Yes --> L[Calculate Discount]
L --> M[Cap Discount ไม่ให้เกินยอดคงเหลือ]
```

```
M --> N[Apply Discount]
N --> O[Add Breakdown]
O --> P{Exclusive หรือ Stop Processing?}
P -- Yes --> Q[Mark Scope as Stopped]
P -- No --> R[Continue Next Promotion]
Q --> R
```







9. Calculation Example ตามโจทย์

Input

Product 1 ราคา 1,000 บาท
Product 2 ราคา 500 บาท

Promotions

Promotion A:
Product 1 ลด 10%
scope = ITEM
priority = 1
stackable = true

Promotion B:

```
Product 2 ลด 100 บาท  
scope = ITEM  
priority = 1  
stackable = true
```

Calculation

```
Product 1:  
original = 1,000  
discount = 1,000 × 10% = 100  
final = 900
```

```
Product 2:  
original = 500  
discount = 100  
final = 400
```

Result

```
Original Total = 1,500  
Discount Total = 200  
Final Total = 1,300
```

10. Calculate vs Confirm Order Flow

ต้องแยกสอง Flow นี้ให้ชัด

Calculate Order

ใช้สำหรับ Preview ราคา

```
User opens cart  
↓  
POST /orders/calculate  
↓  
System calculates price  
↓  
Return price preview
```

ข้อสำคัญ:

ยังไม่ consume promotion usage

Confirm Order

ใช้ตอนสร้าง Order จริง

```
User confirms order
↓
Server re-calculates price
↓
Begin DB transaction
↓
Lock promotion usage
↓
Validate usage limit
↓
Create order
↓
Consume promotion usage
↓
Save calculation snapshot
↓
Commit transaction
```

เหตุผลที่ต้อง Re-calculate:

- Promotion อาจหมดอายุ
- Promotion อาจถูกปิด
- Coupon usage อาจเต็ม
- ราคาสินค้าอาจเปลี่ยน
- Client อาจแก้ Payload

11. Output Breakdown Design

ทุก Promotion ที่ Apply หรือ Skip ควรถูกบันทึก

Applied Promotion

```
{
  "promotion_id": 1,
  "promotion_name": "Product 1 Discount 10%",
  "scope": "ITEM",
  "target": {
    "product_id": 1
  },
  "before_amount": 1000,
  "discount_amount": 100,
  "after_amount": 900
}
```

Skipped Promotion

```
{
  "promotion_id": 3,
  "promotion_name": "Cart Discount 100",
  "reason": "min_order_amount_not_reached"
}
```

เหตุผลที่ต้องมี Breakdown:

- Debug ง่าย
- อธิบายราคาให้ลูกค้าได้
- Support ตรวจสอบย้อนหลังได้
- ใช้เป็น Audit Log
- ใช้เขียน Test Case ได้ชัดเจน

12. Important Rules Summary

Topic	Decision
Calculation Order	ITEM → CART → COUPON → SHIPPING → ROUNDING
Promotion Order	scope_order → priority → created_at → id
Priority	ค่าน้อยกว่าทำก่อน
Stacking	ใช้ <code>stackable</code>
Conflict	ใช้ <code>exclusive</code> และ <code>stop_processing</code>
Calculation Base	คำนวณจากยอดคงเหลือแบบ Sequential
Money Type	ใช้ minor unit เช่น satang
Product Price	โหลดจาก Server เท่านั้น
Calculate Order	Preview เท่านั้น
Confirm Order	ต้อง Re-calculate และใช้ Transaction
Debug	ต้องมี Applied/Skipped Breakdown

13. Summary

Calculation Flow ที่เหมาะสมสำหรับระบบนี้คือ:

```

Validate Request
→ Load Server-side Product Price
→ Build Calculation Context
→ Load Active Promotions
→ Filter Eligible Promotions
→ Sort by Scope and Priority
→ Apply Item Promotions
→ Apply Cart Promotions
→ Apply Coupon Promotions
→ Apply Shipping Promotions
→ Cap Discount
→ Round
→ Generate Breakdown
→ Save Audit Log
→ Return Final Price

```

Flow นี้ตอบโจทย์เพราะ:

- คำนวณ Promotion หลายตัวได้ถูกต้อง

- รองรับ Promotion แบบซ้อนกัน
- ควบคุมลำดับด้วย Priority
- เพิ่ม Promotion ใหม่ได้ผ่าน Strategy
- ป้องกันราคาติดลบ
- ตรวจสอบย้อนหลังได้
- พร้อมต่อยอดไปยัง Production System